

RECUT: implementation and validation plan

Protocol before experiments - version 0.1

16 September 2026. This plan follows the idea and novelty assessment. The aim is a reproducible, bounded prototype with honest go/no-go decisions, not a claim that a complete conference contribution can be guaranteed in one run. No procedure is literally foolproof; independent controls and explicit stop conditions make mistakes detectable.

Phase 1: define the executable contract

Use an unmodified public causal decoder's weights and layer operations, greedy decoding, evaluation mode and no sampling. The pilot covers fixed-length windows even if an EOS token appears; it tests computation recovery, not conversational quality. Pin model revisions and library versions. Keep remote-code execution disabled. Record hardware, precision and execution path.

Let N be the layer count, T the verified prefix length, W the uncommitted decode window and L the known corrupted cache layer. The initial next input token is chosen from clean prefix logits. Inject one predeclared accidental finite perturbation into the cached prefix, then execute W transitions before recovery. The detector/localizer is an oracle in this first pilot; do not count its cost as zero in deployment conclusions.

Save a full trusted prefix checkpoint for all valid methods. Record its bytes and copy cost separately. During the speculative window, save hidden inputs at configured cuts. Replay must use the same single-token shapes and attention computation as the clean reference. Do not silently switch to chunked prefill for one baseline.

Phase 2: conservative algorithm

1. Keep token outputs private until the window passes verification. Store journal entries with token index, layer cut and input token identity.
2. After localization, choose the deepest saved cut $c \leq L$. If none exists, $c = 0$ and use ordinary replay. Reject cuts above L and reject missing or inconsistent journal provenance.
3. Restore the verified prefix for affected layers from the trusted checkpoint. Discard or overwrite their derived suffix. Lower layers may retain their old suffix only while the replay input tokens agree.
4. At each transition, replay from cut c using the corresponding saved hidden input. Compare the corrected greedy decision with the speculative decision. After the first disagreement, run all remaining transitions from the embedding layer using corrected tokens. Never use future stale journals after that disagreement.
5. Verify complete final cache state, logits, token sequence and continuation against an independently executed full replay. Fail closed on unsupported architecture, non-finite state, uncertain localization, missing checkpoint or numerical mismatch. The pilot records these failures rather than claiming an operational verifier.

2. Baselines and test matrix

Method	Role and fairness condition
Clean reference	Same model, precision, incremental shapes and starting prefix; no perturbation.
Faulty / no recovery	Measures the effect of the benign perturbation; not an attack-success metric.
Slot-only restore	Replace the original damaged prefix location but retain derived suffix; negative control for incomplete recovery.
Full suffix replay	Main valid baseline: restore the same verified prefix and recompute W transitions through all layers.
Sparse RECUT	Cuts at coarse layer fractions; deepest clean cut, full-layer fallback at divergence.
All-layer journal	Provenance-aware activation-recovery upper bound: cut exactly at L. Pay every saved activation byte; do not claim this is the published HCache implementation.
Unsafe cut control	Deliberately pick an input above L in tests; it must not be admitted by the production policy.

Workloads and perturbations

Begin with SmolLM2-135M and Qwen2.5-0.5B: different GQA configurations and layer counts. Use small deterministic constructed prompts for mechanism validation, explicitly not a standard quality benchmark. Include varied prose, arithmetic, structured records and code-like text without executing generated content. Longer-context and real benchmark suites are later thesis requirements.

Debug trials use separate seeds and prompts. Freeze the actual executable configuration after debugging and before the primary run; record deviations from this prospective plan. Primary axes: early/middle/late layers, K versus V, short versus longer detection windows and scalar versus multi-coordinate finite perturbations. Sample coordinates without searching for harmful outputs. Include clean/no-op controls.

No hardware errors, malicious prompt attacks, external targets, instruction-level instrumentation, NVBit or changes to the existing AWS Kubernetes cluster are authorized by this protocol. This is tensor-level numerical reliability testing on a dedicated task allocation.

Unit and integration tests

Test no-fault identity, baseline replay identity, $c=0$ equivalence, rejection of $c>L$, missing journal fallback, fault at first/last layer, $W=1$, no token divergence, immediate and delayed token divergence, preservation of clean prefix and lower-layer cache, and full-state equality after recovery. Compare custom stepping against the library's own incremental forward path. Independent code review must look for shared-oracle bugs and comparisons to the wrong reference.

3. Measurements and analysis

Correctness first

For every episode, retain model revision, prompt, seed, injection coordinates/value change, precision, layer, window, method and result. Compare every recovered K/V tensor using exact equality under the fixed execution graph; also record maximum absolute error as a diagnostic. Token equality alone is insufficient. Check a short fault-free continuation after the repaired window. A single unexplained failure invalidates an unconditional exact-recovery claim.

The clean reference and repair paths share model operations, so agreement alone is not independent verification. Add a library-forward equivalence control and algorithm-specific adversarial unit cases using benign numerical inputs. Do not report zero failures as a zero population risk: with n independent episodes and zero failures, the one-sided 95% binomial upper bound is $1 - 0.05^{(1/n)}$, conditional on the sampling model. Correlated repeated prompts reduce effective evidence; report this limitation.

Performance and resource accounting

Time recovery with GPU synchronization and warmup; alternate method order across repeated paired measurements. Report medians and distributions, not a cherry-picked best case. Include restoration work within the timed repair boundary; disclose setup excluded equally from all methods. Count executed layer-token steps as a platform-independent work diagnostic, not a substitute for wall time.

Measure a no-fault path with no journal, sparse journals and all-layer journals. Report checkpoint copy time, journal write overhead, journal bytes, complete checkpoint bytes, peak allocated device memory and private-window latency. In GQA models a hidden activation can exceed that layer's KV bytes; no blanket memory-saving claim is allowed.

Separate recovery speed from service benefit. If journal overhead per window is H and saved recovery time per incident is S , the simple single-incident expected-time break-even probability is $p > H/S$, when $S > 0$. This is a diagnostic inequality, not an estimate of real fault frequency. Add common verification/checkpoint costs to end-to-end accounting; holdback latency is a separate objective even if the inequality is favorable.

Predeclared decisions

Go for the correctness mechanism only if valid methods exactly recover all supported frozen primary cases and negative controls demonstrate the expected gap. Go for a performance thesis only if sparse cuts offer useful Pareto points and repeated measured gains survive overhead accounting. A recovery-only median speedup of at least 1.2x is a pilot signal, not proof of deployment value; a loss, ties or high break-even rate must be reported.

Do not tune cut placement or fault severity on the final trial outcomes. If a bug requires a change, archive the failed run, document it and use a separately identified corrected run. Exploratory follow-ups remain labeled exploratory.

4. Execution, deliverables and publication gates

Safe HPC execution

Use one Slurm-allocated RTX 5000 Ada GPU, four CPU cores and initially 16-24 GB host RAM. First allocation limit: up to two hours, with early release when work completes. Install only into the new task's environment on the allocated node. Fetch public model weights without storing account passwords. Keep all credentials out of code, reports and command logs. Do not run compute on the login node, reserve multiple workstations, or use binary GPU instrumentation.

Preserve all earlier RAVEN and ProofOps evidence. New files live in research-next locally and a separate RECUT directory on Lustre. Job IDs, manifests and stdout/stderr belong in the artifact. Stop runaway work on errors, disk pressure or scheduler limits; do not delete unrelated jobs or files. No AWS cluster changes, external services or publication submission.

Deliverables in order

1. Idea/novelty PDF, followed by this implementation-plan PDF, created before experiment code.
2. Readable source, pinned requirements, deterministic trial configuration, unit/integration tests and commands for reproducing the pilot.
3. Raw per-episode results, timing repetitions, environment and model manifests, automatic summary, independent audit and checksums. Preserve failed/debug runs separately.
4. Professor-facing final draft PDF distinguishing measured findings from proposed extensions; include limitations and a candid publication assessment. Include the 100-paper screening ledgers and website-access notes in the supporting package.

What a full thesis still needs

After a successful pilot: formalize the recovery theorem; implement a realistic verified-window detector and charge its cost; test localized and uncertain/multiple faults; evaluate 1-8B models, long contexts, continuous batching and sampling with coupled RNG; compare optimized serving implementations and modern activation restoration; study checkpoint reliability and real application requirements. These are research extensions, not hidden claims of completion.

What would support A/A*-level consideration

An independently validated, general recovery result; a material contribution beyond HCache/HybridServe/KVPR and delayed-error checkpointing; realistic workloads and faults; several scales and architectures; strong systems baselines; honest end-to-end cost; reproducible artifacts; and a clear explanation of where simpler verification wins. A small custom-loop demonstration alone is not sufficient. Ranking and current venue rules must be checked when selecting a target.

References and uncertainty

The companion idea PDF contains the primary-source novelty matrix. Especially relevant are [HCache](#), [HybridServe](#), [KVPR](#), [delayed silent-error checkpointing](#), [vLLM valid-prefix recovery](#) and [stage-replay numerical divergence](#). Their mechanisms constrain novelty; this plan is not a claim of superiority over their complete implementations.